PaperDatabase : une base de connaissance textuelle et vérifiable pour la lecture assistée d'articles scientifiques

Léonard Rivals Maksym Lytvynenko

3 octobre 2026

Résumé

Préparer une thèse suppose de lire, comparer et citer des dizaines d'articles. Un grand modèle de langage (LLM) peut aider, mais il travaille mal sur des PDF bruts : il en relit le texte intégral à chaque question, ne sait pas où chercher un chiffre et mélange ce que dit un article avec ce qu'il en déduit. PaperDatabase convertit chaque PDF en fichiers texte structurés (métadonnées, sections paginées, références, fiche de lecture, tableau de benchmarks) qu'un agent LLM, ici Claude Code, parcourt du résumé vers le détail. Une couche de recherche de passages, de type RAG, s'y ajoute : index lexical BM25 et index dense (bge-small-en-v1.5) fusionnés par Reciprocal Rank Fusion, avec reranking optionnel. Cette couche ne génère rien. Elle renvoie des passages identifiés, paginés et rejouables, ainsi qu'une commande qui contrôle qu'une citation, et en particulier ses nombres, figure bien dans le passage invoqué. La rédaction reste à l'agent, encadrée par des règles de sourçage. Chaque réponse de l'agent s'ouvre par un verdict de suffisance, qui confronte les exigences de la question aux passages lus. Un banc d'évaluation reproductible, sans LLM ni réseau, mesure l'extraction, la recherche, la vérification, les temps et, depuis peu, les réponses rédigées. Sur 101 articles (8 601 passages), la recherche hybride avec reranking obtient un recall@5 de 0,72 sur un jeu de questions réservé, qui n'a servi à aucun réglage : c'était 0,80 sur 45 articles, et la cause de cette baisse, corpus ou code, n'est pas encore établie. En français, le meilleur mode sur CPU retrouve 4 questions sur 8 ; un moteur optionnel sur GPU (Qwen3) les retrouvait toutes sur un corpus plus petit. La vérification refuse les citations dont un chiffre a été modifié, à une exception près, documentée. Aucun score de recherche ne détecte de façon fiable qu'une question sort du corpus, mais l'agent, guidé par le verdict de suffisance, reconnaît les 6 questions hors corpus de l'évaluation des réponses sans en déclarer aucune autre absente à tort, et au second run toutes ses citations sont acceptées. Un essai réel de réponse sourcée a consommé environ 2 500 tokens, alors qu'un seul texte intégral en représente environ 12 000 en médiane, et déléguer trois fiches de lecture à des sous-agents a coûté 41 % de moins que de les rédiger en direct (un seul essai). Le même code sert deux autres bases, sur la vision embarquée sur drone (87 articles), où la recherche tient son seuil, et sur l'apprentissage continu pour la maintenance prédictive (56 articles).

Table des matières

1 Introduction **3**

2 Principes de conception **4**

3 Architecture et pipeline **4**
3.1 Acquisition et ingestion 4
3.2 Modèle de données 5
3.3 Plusieurs bases 6
3.4 Modules 7

4 La couche de recherche (RAG) **7**
4.1 Un RAG sans génération 7
4.2 Découpage en passages 7
4.3 Deux index et une fusion 8
4.4 Commandes 9
4.5 Protocole de réponse : la skill `/paper-ask` 10
4.6 Un moteur optionnel sur GPU et l'acquisition open access 11

5 Ce que le logiciel apporte **12**

6 Tests et évaluation **14**
6.1 Tests unitaires 14
6.2 Le banc d'évaluation `pdb eval` 14
6.3 Qualité de l'extraction 15
6.4 Recherche de passages 15
6.5 Détecter qu'une question sort du corpus 17
6.6 Vérification des citations 17
6.7 Temps d'exécution 18
6.8 Essai réel de bout en bout 18
6.9 Évaluer la réponse rédigée 18
6.10 Un second corpus : la base drone 19
6.11 Banc synthétique (`pdb synth`) 20
6.12 Menaces à la validité 20

7 Limites et perspectives **21**

8 Conclusion **22**

A Glossaire des acronymes **24**

B Lexique **25**

C Commandes **27**

1 Introduction

Contexte. Ce travail est né d'un besoin concret : préparer un projet de thèse sur la vision embarquée sur FPGA (détecteurs YOLO, autoencodeurs, apprentissage continu). Le domaine publie beaucoup de chiffres (images par seconde, GOPS, watts, ressources LUT, DSP et BRAM), mesurés dans des conditions hétérogènes. Les comparer demande de savoir d'où vient chaque valeur. Le corpus de ce cas d'usage sert de banc d'essai, mais l'outil ne dépend pas du domaine : le même code sert aujourd'hui deux autres bases, l'une sur la vision embarquée sur drone, l'autre sur l'apprentissage continu embarqué sur microcontrôleur pour la maintenance prédictive. Ce qui dépend du sujet (axes, règles de classement, tags, périmètre) est déclaré dans les fichiers de chaque base.

Problème. Un agent LLM à qui l'on donne des PDF a trois défauts :

1. *coût* : il relit des textes intégraux (environ 12 000 tokens en médiane) pour répondre à une question qui tient en trois phrases ;
2. *traçabilité* : un chiffre ou une citation ne porte pas l'endroit exact d'où il vient, et une citation reformulée passe pour textuelle ;
3. *confusion entre lu et déduit* : l'agent comble les manques avec ses connaissances générales, sans le signaler.

Approche. PaperDatabase ne cherche pas à remplacer l'agent. Il lui fournit une base où chaque information porte sa source, et des commandes déterministes qui répondent aux questions factuelles sans appel à un LLM. La génération se limite aux questions qui demandent une synthèse, et elle se fait à partir de sorties déjà sourcées.

Contributions.

— Un pipeline d'ingestion PDF → texte structuré (GROBID, repli PyMuPDF, réparation des ligatures, reconstruction des tableaux), idempotent, qui ne devine jamais une métadonnée absente (section 3).
— Un modèle de données entièrement textuel, validé à chaque réindexation, où toute valeur numérique porte sa section source et un niveau de confiance (section 3.2).
— Une couche de recherche hybride sans génération, organisée en trois niveaux de coût et dotée d'une vérification automatique des citations (section 4).
— Un banc d'évaluation reproductible, sans LLM ni réseau, qui mesure la qualité de l'extraction, de la recherche et de la vérification des citations, ainsi que les temps, avec un jeu de questions réservé qui n'a servi à régler aucun paramètre ; et une suite de tests unitaires qui s'exécute sans modèle (section 6).
— Un protocole de réponse qui pose les exigences d'une question avant de chercher et ouvre chaque réponse par un verdict de suffisance (`suffisant`, `partiel`, `absent`), et une évaluation des réponses rédigées elles-mêmes, notées sans LLM (sections 4.5 et 6.9).
— Des preuves atomiques (`claims.jsonl`), chacune liée à un passage et contrôlée à la réindexation, et des conditions de comparaison explicites dans le tableau de benchmarks (section 3.2).
— Un moteur de recherche optionnel sur GPU (Qwen3), intégré à la même commande, et une acquisition d'articles open access validée par l'utilisateur (section 4.6).
— Une séparation du code et des données qui permet de tenir plusieurs bases isolées, chacune avec son profil, ses règles et son banc d'évaluation, et d'échanger des articles entre elles (section 3.3) ; le banc est reproduit sur un second corpus (section 6.10).
— Un journal du coût des sous-agents auxquels l'agent principal délègue des tâches, et une première mesure appariée de ce que la délégation fait gagner (section 5).

2 Principes de conception

Le consommateur final de la base n'est pas un moteur de recherche. C'est un agent LLM qui dispose déjà d'outils de lecture de fichiers et de recherche textuelle (`grep`, `read`). Sur un corpus de quelques dizaines à quelques centaines d'articles, une arborescence bien nommée, un index et un CSV filtrable suffisent pour l'essentiel des questions, et restent lisibles, comparables d'une version à l'autre et réparables à la main. Quatre règles en découlent.

1. **Tout est du texte.** Pas de base de données ni de format binaire, à part le PDF d'origine, conservé comme preuve. L'index de recherche (section 4) est un cache dérivé, reconstructible, hors de l'arborescence versionnée.
2. **Chaque chiffre est traçable.** Toute valeur du tableau de benchmarks porte la section, la table ou la figure dont elle vient. Une valeur non rapportée par les auteurs s'écrit `n/r`, jamais une estimation.
3. **Lecture du résumé vers le détail.** L'agent lit la couche la plus haute qui répond, et ne descend que si nécessaire (tableau 1). Il ne rouvre jamais le PDF.
4. **Le schéma est codé à un seul endroit** (`lib/schema.py`) et vérifié à chaque réindexation, qui s'arrête au premier écart.

Table 1 – Couches de lecture, du résumé au détail.

Couche	Fichier ou commande	Sert à
1	`INDEX.md` (généré)	savoir quels articles existent, par axe, fichés ou non
2	`benchmarks.csv` via `pdb bench`	trouver un chiffre, avec sa section source ; `--comparable` : savoir quelles lignes se comparent
3	`papers/<id>/fiche.md`	lire l'analyse critique d'un article
3 bis	`papers/<id>/claims.jsonl`	retrouver une affirmation de l'article, déjà liée à son passage
4	`pdb search`, `show`, `verify`	trouver et citer les 2 ou 3 passages qui répondent
–	`fulltext.md`	texte intégral, en dernier recours

Un fichier d'instructions (`CLAUDE.md`) décrit à l'agent cet ordre de lecture et des règles de travail : ne jamais inventer de référence, écrire `n/r`, séparer ce qui est lu de ce qui est déduit, signaler les preprints et les métriques non comparables.

3 Architecture et pipeline

3.1 Acquisition et ingestion

`bin/add.py` récupère un article arXiv (métadonnées par l'API, puis le PDF), résout un DOI vers sa version arXiv par OpenAlex (Priem et al., 2022), ou copie un PDF local. Les articles sous paywall se déposent à la main : l'outil ne contourne aucun accès.

`bin/ingest.py` envoie chaque PDF à GROBID (GROBID contributors, 2008–2026), qui renvoie une structure TEI (en-tête, sections, figures, bibliographie). En l'absence de GROBID, une extraction de repli par PyMuPDF découpe le texte par heuristique sur les titres numérotés, et le signale dans les métadonnées. Le pipeline applique ensuite, dans l'ordre :

— *fusion des métadonnées* par ordre de confiance : export de citation de l'éditeur, puis arXiv, puis GROBID. Un champ inconnu reste `null` ;

— *nettoyage* : en-têtes et pieds répétés, numéros de page, bibliographie sortie du corps et rangée dans `refs.jsonl` ;

— *réparation des ligatures perdues* (« classi cation » pour « classification ») : un mot n'est recollé que si le résultat est connu du corpus ou d'une liste intégrée, et que le fragment ne l'est pas ;

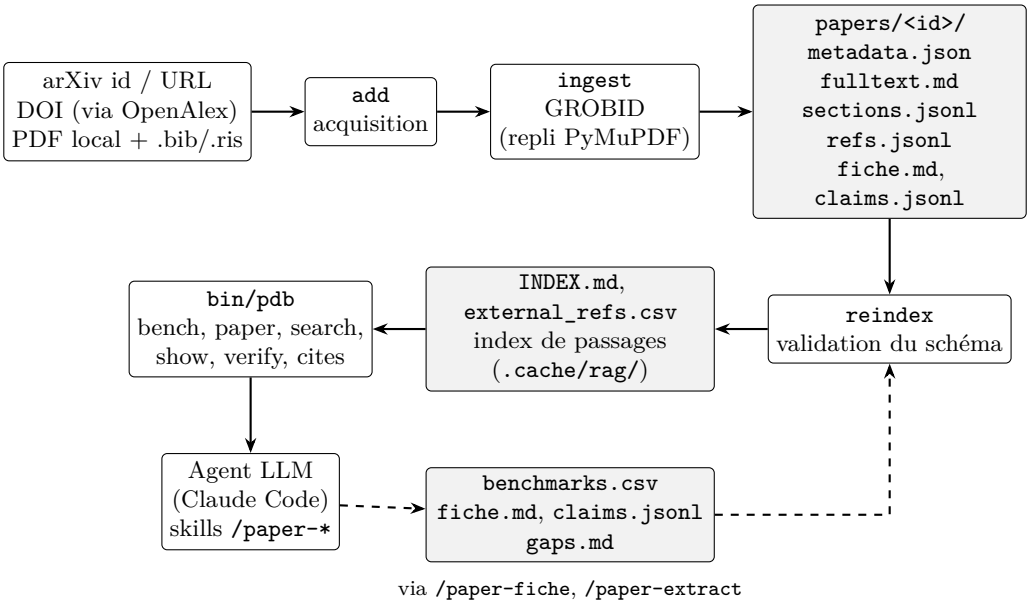


FIGURE 1 – Flux de données. Les flèches pleines sont automatiques ; les flèches en tirets passent par l'agent, qui rédige les fiches, les preuves atomiques et les lignes de benchmark à partir du texte extrait.

— *tableaux* : un tableau dont GROBID n'a gardé que la légende passe par deux replis successifs. PyMuPDF le cherche d'abord sur sa page (tableaux à filets). Docling (Auer et al., 2024) vient ensuite, sur demande (`ingest --reparse --tables docling`), avec une reconnaissance optique (OCR) en option pour les tableaux publiés en image. Un corps reconstruit est marqué « reconstruit par PyMuPDF / Docling : à vérifier », et un tableau resté vide est signalé par `pdb status`. Chaque tableau garde sa page, son cadre sur la page (`bbox`) et l'extracteur qui l'a produit (`table_source`). Sept signaux de qualité, calculés à la lecture, repèrent les tableaux suspects : cellules vides, nombre de cellules variable d'une ligne à l'autre, absence d'en-tête, caractères douteux d'OCR, nombres collés, colonnes perdues, légende appariée par rang plutôt que par numéro (`pdb status --tables`) ;
— *paragraphes mal classés* : un paragraphe que GROBID a pris pour une figure est rendu au corps quand il ressemble à de la prose.

L'ingestion est idempotente, et une ré-extraction (`--force`, ou `--reparse` depuis les TEI conservés) n'écrase jamais une fiche de lecture déjà rédigée ni une classification corrigée à la main.

3.2 Modèle de données

Chaque article devient un dossier `papers/<paper_id>/`. L'identifiant est stable et lisible (`arxiv-<id>-<slug>` ou `<année>-<auteur>-<slug>`). Le texte extrait existe sous deux formes : `fulltext.md`, lisible, et `sections.jsonl`, une ligne par section avec son type (résumé, section, figure, table) et sa page. Cette seconde forme est la matière première de l'index de passages.

Le fichier `benchmarks.csv` contient une ligne par configuration rapportée : un article qui présente trois variantes produit trois lignes. Outre les colonnes de mesure (modèle, plateforme, quantification, FPS, GOPS, latence, puissance, LUT, DSP, BRAM, jeu de données, toolflow), trois colonnes servent à juger chaque valeur :

— `source_section` : section, table ou figure d'origine ;
— `confidence` : `high` (lu dans une table), `medium` (lu dans le texte) ou `low` (déduit, à revérifier) ;
— `power_scope` : portée de la puissance rapportée (`chip`, `board`, `board-dynamic`, `estimated`, `unspecified`). Deux puissances de portées différentes ne se comparent pas : une carte peut

consommer plusieurs watts au repos, qu'une mesure « carte » inclut et qu'une mesure « puce » exclut.

Ces vingt colonnes ne disent pas tout des conditions d'une mesure. Sept colonnes *optionnelles*, ajoutées à la demande (`pdb bench --add-columns`), les complètent : le passage qui contient la valeur (`chunk_id`), la définition exacte de la métrique, le split évalué, la résolution d'entrée, le batch, le protocole de mesure et la valeur telle qu'écrite quand elle a été convertie. Absente, une colonne optionnelle vaut `n/r` : une base existante reste valide sans migration, et aucune ligne actuelle ne les renseigne encore. `pdb bench --check` vérifie que chaque nombre d'une ligne qui cite un passage figure bien dans ce passage, et `pdb bench --comparable` regroupe les lignes par tâche, jeu de données, split et métrique (ou portée de la puissance) avant toute comparaison entre articles. Il signale les conditions qui varient dans un groupe, celles qui manquent, et les comparaisons impossibles. Sur la base fpga, les 56 lignes de détection sur COCO, issues de 7 articles, forment 9 groupes, dont aucun ne réunit deux articles : aucune de ces valeurs n'est directement comparable à une autre.

Les fiches de lecture (`fiche.md`) suivent un gabarit fixe (problème, méthode, résultats, limites, pertinence) avec une section « statut de lecture » obligatoire qui sépare le lu du déduit. Elles sont rédigées par l'agent (commande `/paper-fiche`) et relues par l'utilisateur.

Une fiche résume ; ses chiffres portent leur source, mais pas ses affirmations qualitatives (méthode, limite, hypothèse). Le fichier `claims.jsonl` d'un article contient ses preuves atomiques : une affirmation par ligne, avec son type (résultat, méthode, limite, hypothèse, contexte), sa voix (les auteurs, ou une déduction du lecteur), le passage source, l'extrait exact qui la porte, et en option ses conditions et la ligne de benchmark qui la transcrit. La skill `/paper-extract` les rédige. À chaque réindexation, chaque extrait est contrôlé par `verify` contre son passage : un extrait absent ou aux nombres faux est une erreur, et la réindexation échoue. Un nombre de l'affirmation absent de la section source est signalé. Premier essai, sur l'article FINN : 32 preuves, 32 extraits vérifiés, une seule note, légitime (valeur tirée d'un passage voisin) ; une erreur introduite exprès (nombre altéré, mauvais passage) est refusée. Les autres articles n'ont pas encore été traités, et la recherche ne lit pas encore ces preuves.

3.3 Plusieurs bases

Le même code sert plusieurs bases de connaissance, une par sujet. Le dépôt en contient trois (tableau 2) : la base *fpga* du cas d'usage, qui sert de référence et porte désormais sur une thèse en IA embarquée sûre et certifiable sur FPGA ; une base *drone* (vision embarquée sur drone : détection, accélérateurs, co-design calcul, énergie et vol), créée ensuite sur un sujet voisin ; et une base *stage*, sur l'apprentissage continu embarqué sur microcontrôleur (MCU) pour la maintenance prédictive, construite à partir de la bibliographie d'un stage. Cette dernière est la plus éloignée du domaine d'origine : ses articles fondateurs (Mahalanobis, 1936, numérisé et passé par l'OCR) n'ont aucune mesure matérielle, et ses benchmarks portent sur des jeux de données de roulements et de turbines plutôt que sur des accélérateurs. La racine du dépôt ne contient que le code. Chaque base est un dossier `bases/<nom>/` qui a ses articles, son `benchmarks.csv`, sa carte des lacunes, son index de passages et ses caches ; les commandes, l'environnement Python et les skills de l'agent y sont des liens vers le code.

TABLE 2 – Les trois bases au 03/10/2026 (sortie de `pdb base list` et `pdb status`).

Base	Articles	Fiches faites	Lignes de benchmark	Passages indexés
fpga (référence)	101	101	299	8 601
drone	87	86	352	5 361
stage	56	56	100	3 640

Ce qui dépend du domaine tient dans deux fichiers par base. `base.json` déclare les axes de veille et des règles, en expressions régulières, qui classent un article dans un axe, lui attribuent des

tags et décident s'il entre dans le sujet (ce dernier test sert à la découverte). `profile.md` décrit en texte le thème et le périmètre (inclus, exclu) ; la skill `/base-profile` propose de l'affiner à partir des fiches, et n'écrit rien sans accord. Chaque base a aussi son `CLAUDE.md`, qui porte les règles de travail propres au domaine : pour la base drone, préciser la gamme de puissance d'un résultat, ou distinguer une détection sol → air d'une détection air → air. Les colonnes de `benchmarks.csv` restent communes à toutes les bases ; les colonnes optionnelles (section 3.2) s'ajoutent base par base. Les figures de `pdb plot` s'adaptent aussi : un bloc `plot` de `base.json` déclare les tâches et familles de plateformes à distinguer et les figures à omettre. La base stage, sans mesures matérielles ni jeux de questions, a 8 figures, contre 16 pour fpga. `pdb plot --check` propose ce bloc d'après le contenu de la base, et `reindex` régénère les figures devenues périmées.

La base active se déduit du contexte : la variable `$PDB_ROOT` (fixée par `pdb --base <nom>`), sinon `$PDB_BASE`, sinon le premier dossier contenant `base.json` en remontant depuis le répertoire courant, sinon la base par défaut d'un registre local (`~/.config/pdb/bases.json`), sinon la base de référence. Deux sessions de l'agent ouvertes dans deux bases travaillent donc chacune sur la sienne.

Les bases restent séparées, mais un article peut passer de l'une à l'autre. `pdb base scan` cherche dans les autres bases les articles utiles à la base courante : ceux qu'elle cite déjà sans les avoir, ceux qui correspondent à ses règles, et les références citées des deux côtés. `pdb base import` copie un article sans le réextraire et remet sa fiche à relire, car sa pertinence dépend du sujet de la base qui le reçoit.

3.4 Modules

Toutes les commandes passent par un point d'entrée unique, `bin/pdb`. Leur sortie suit les mêmes conventions : TSV par défaut, une ligne d'en-tête sans bannière, des notes préfixées par `#`, des identifiants courts d'articles (`1612.07119`, `2022-feng`) avec leur légende en pied, et un code de sortie qui distingue « des résultats », « aucun résultat » et « erreur d'argument ».

4 La couche de recherche (RAG)

4.1 Un RAG sans génération

Dans un système de *Retrieval-Augmented Generation* (Lewis et al., 2020), le LLM reçoit, avec la question, les passages d'un corpus qui y répondent, et rédige à partir d'eux. PaperDatabase sépare les deux moitiés. Les commandes `bin/pdb` font la recherche, de façon déterministe et sans appel à un LLM. L'agent les appelle, lit leurs sorties et ne rédige que si une synthèse est nécessaire. Ce choix n'ajoute ni clé d'API ni coût : la génération est faite par l'agent déjà en service.

Trois objectifs guident la conception, par ordre de priorité : la *vérifiabilité* (tout élément renvoyé porte une source rejouable), l'*économie de tokens* (répondre sans lire de texte intégral), et l'*accès par le sens* (retrouver « continual learning » en cherchant « apprentissage incrémental », ce que `grep` ne sait pas faire).

Une sortie N0 ou N1 est souvent déjà une réponse : un tableau filtré ou trois passages sourcés suffisent à une question factuelle, et l'agent peut les transmettre sans les reformuler. Le niveau N2 n'ajoute pas d'information : il relie ce que N0 et N1 ont renvoyé, et il étiquette ses inférences comme telles.

4.2 Découpage en passages

L'unité de départ est une entrée de `sections.jsonl`, qui respecte déjà les paragraphes. Une entrée de plus de 250 mots est coupée en fenêtres de 200 mots avec 40 mots de recouvrement, parce que les modèles d'embeddings tronquent au-delà de 256 à 512 tokens. Chaque passage reçoit un identifiant rejouable, `<section>.<k>` (par exemple `1612.07119#019.3`), et garde son article, sa section, sa page et l'empreinte SHA-1 de son texte. Le texte indexé est préfixé du titre de

TABLE 3 – Principaux modules de `lib/`. L'ensemble est écrit en Python ; seules la recherche dense, le moteur GPU, les figures et la visualisation ont des dépendances optionnelles.

Module	Rôle
`tei.py`, `doc.py`, `pdftext.py`	lecture de la sortie GROBID, représentation commune, repli PyMuPDF
`ligatures.py`, `tables.py`, `docling_tables.py`	réparation des ligatures, reconstruction des tableaux (PyMuPDF, puis Docling), signaux de qualité
`citation.py`, `openalex.py`	exports `.bib`/`.ris`, client OpenAlex avec cache
`schema.py`	validateurs du modèle de données, seul endroit où il est codé
`refs.py`	rapprochement des références (arXiv, DOI, titre normalisé), citations internes
`query.py`	requêtes structurées sans LLM (`bench`, `paper`, `cites`), `--check`, groupes comparables
`rag.py`	passages (tableaux découpés par rangées), BM25, index dense, fusion, reranking, vérification des citations ; moteur Qwen optionnel
`claims.py`	contrôle des preuves atomiques (`claims.jsonl`) contre leur passage
`rag_qwen.py`, `rag_models.py`	moteur GPU : modèles Qwen3 épinglés (partagés avec `pdb search`), index de preuves FTS5 de `pdb rag`
`rag_external.py`	acquisition open access par étapes (OpenAlex, arXiv), import après accord
`evaluation.py`	banc d'évaluation (`pdb eval`) : extraction, recherche, `verify`, temps, réponses finales
`agentlog.py`, `agentcost.py`	journal des sous-agents, coût calé sur Claude Code, économie de la délégation
`status.py`	état de la base et prochaines actions
`base.py`, `registry.py`	base active, lecture de `base.json`, classement selon ses règles ; registre des bases
`bases.py`, `xbase.py`, `vizbases.py`	création et contrôle des bases, scan et import entre bases, onglet Bases de la visualisation
`plotdata.py`, `plotfig.py`, `plotrun.py`	figures statiques (`pdb plot`) : benchmarks, évaluation, corpus ; régénération des figures périmées
`graph.py`, `lineage.py`, `vizfig.py`, `embed.py`	visualisation 3D et généalogie (outil secondaire)

section, ce qui donne du contexte sans modifier le texte renvoyé. Un tableau de plus de 250 mots est découpé autrement : par rangées entières, chaque fenêtre répétant la légende abrégée et la ligne d'en-tête, pour qu'une valeur ne soit jamais séparée de son unité ni de sa colonne. Sur les trois bases, 181 tableaux sont concernés ; ils donnent 610 fenêtres (médiane 186 mots) sans perte de mot. La base fpga compte aujourd'hui 8 601 passages.

4.3 Deux index et une fusion

Index lexical. Un BM25 (Robertson and Zaragoza, 2009), écrit à la main ($k_1 = 1{,}5$, $b = 0{,}75$, jetons alphanumériques en minuscules), retrouve les termes exacts : noms de cartes (`ZCU104`), de modèles (`YOLOv5n`), sigles et nombres.

Index dense. Le modèle `BAAI/bge-small-en-v1.5` (Xiao et al., 2024) (384 dimensions), chargé par sentence-transformers (Reimers and Gurevych, 2019), encode chaque passage en un vecteur normalisé. Il retrouve les reformulations et, dans une certaine mesure, les requêtes dans une autre langue. La requête reçoit le préfixe d'instruction recommandé par le modèle ; les passages, non.

TABLE 4 – Trois niveaux de requête, du moins cher au plus cher. On ne monte d'un niveau que si le précédent ne suffit pas. Coûts relevés sur le corpus actuel.

Niveau	Source	LLM	Coût typique
N0 – requête structurée	`benchmarks.csv`, métadonnées	non	100–500 tokens
N1 – recherche de passages	index de passages	non	400–800 tokens, +300 par passage ouvert
N2 – synthèse	sorties de N0 et N1 seulement	oui	celui de la rédaction

Fusion. Les deux classements sont fusionnés par Reciprocal Rank Fusion (Cormack et al., 2009) :

$$\text{score}(d) = \sum_{i \in \{\text{BM25, dense}\}} \frac{1}{60 + \text{rang}_i(d)}. \tag{1}$$

Cette méthode ne dépend pas de l'échelle des scores, ne demande aucun réglage et reste déterministe. En cas d'égalité, l'identifiant le plus petit passe en premier : la même requête sur le même index donne toujours le même classement.

Pertinence absolue. Le score RRF est un rang, pas une mesure de pertinence : `search` renvoie toujours k passages, même quand la base ne traite pas le sujet. La sortie affiche le cosinus entre la requête et chaque passage. Une première version avertissait (`# pertinence faible`) quand le meilleur cosinus passait sous un seuil calibré à la main sur quelques requêtes. Le banc d'évaluation a montré que cette alerte ne se déclenchait jamais, et qu'aucun seuil, ni sur le cosinus ni sur le score d'un cross-encoder, ne séparait les questions hors corpus des autres (section 6.5). Elle a donc été retirée, parce qu'elle rassurait à tort : c'est l'agent qui juge, à la lecture des extraits, si la base répond, et qui le dit sinon au lieu de combler le vide.

Reranking (option). `--rerank` reclasse les 30 premiers résultats avec le cross-encoder `ms-marco-MiniLM-L-6-` un MiniLM (Wang et al., 2020) entraîné sur MS MARCO (Nguyen et al., 2016) selon l'approche de Nogueira and Cho (2019). Il coûte environ 1,4 s par recherche à chaud. Il n'est pas activé par défaut, pour les raisons données en section 6.4.

Stockage. Les vecteurs sont rangés dans `.cache/rag/<modèle>.npz`, indexés par la clé `chunk_id:sha1`. La mise à jour est incrémentale : seuls les passages nouveaux ou modifiés sont encodés (environ 8 min pour une construction complète des 8 601 passages de la base fpga sur CPU, soit 18 passages par seconde ; 0,2 s sans changement). Les passages et le BM25 sont recalculés à chaque commande en moins d'une seconde. Sans sentence-transformers, ou si l'index est périmé, `search` passe en lexical seul et le signale. À cette échelle, une recherche exhaustive en numpy prend quelques millisecondes : une base vectorielle dédiée n'apporterait rien avant de l'ordre de 10^5 passages (estimation, non mesurée).

4.4 Commandes

`pdb bench` filtre `benchmarks.csv` (sous-chaînes, bornes numériques, colonnes renseignées, tri, agrégation par `--count`). Les colonnes de source (`source_section`, `confidence`, `power_scope`) restent affichées même si on ne les demande pas, et une ligne de confiance basse est préfixée par `!`.

```
$ bin/pdb bench --platform kv260 --cols model,quantization,fps,power_w
paper       model                  quantization  fps  power_w  power_scope  source_section  confidence
2507.18174  YOLOv5 (variante n/r)  INT8          9    3.5      unspecified  Table I         high
# ids: 2507.18174=arxiv-2507.18174-real-time-object-detection-classification
```

`pdb search` renvoie une ligne par passage : identifiant, section, page, cosinus et un extrait d'environ 160 caractères centré sur le terme le plus rare de la requête (termes rares entre guillemets). Sortie réelle, extraits raccourcis :

```
$ bin/pdb search "power measured at the wall versus the FPGA rails" -k 3
chunk              section                       page  cos   snippet
1612.07119#022.0   5. EVALUATION / 5.2.2. ...     8     0.78  ...both the «wall» «power» and «FPGA» «power
    ↪ » readings...
2025-hozhabr#051.1 V. RESULTS ANALYSIS ... / B. ...  31    0.77  ...computational «power», or throughput, «
    ↪ measured» in...
1612.07119#019.3   5. EVALUATION / 5.1. ...        7     0.73  ...PMBus interface to monitor the «FPGA» «
    ↪ power» supply «rails»...
```

`pdb show` affiche le texte exact d'un passage, avec son en-tête (section, page, début du SHA-1), et, sur demande, ses voisins. `pdb verify` contrôle qu'une citation figure dans le passage invoqué. Les deux textes sont normalisés (ligatures, casse, césures, tout ce qui n'est ni lettre ni chiffre), puis comparés par recherche exacte ou, à défaut, par la meilleure fenêtre de `difflib`. Le verdict est l'un des cinq suivants :

- `OK` : la citation figure telle quelle, après normalisation ;
- `APPROX` : ressemblance d'au moins 0,85 (paraphrase proche) ;
- `NOMBRES` : ressemblance d'au moins 0,85, mais les nombres de la citation ne sont pas ceux du passage ;
- `ABSENT` : ressemblance inférieure à 0,85 ;
- `MODIFIÉ` : le passage a changé depuis que son SHA-1 a été relevé.

Seuls `OK` et `APPROX` donnent un code de sortie nul. Le statut `NOMBRES` est issu du banc d'évaluation. Dans une phrase longue, un chiffre changé laisse la ressemblance au-dessus du seuil, et la première version acceptait ainsi toutes les citations dont un chiffre avait été modifié (section 6.6). Le contrôle ajouté exige que les nombres de la citation forment une suite contiguë des nombres du passage. Chacun doit y figurer avec ses mots voisins : « Section 3 reviews » est refusé même si le passage contient un 3 ailleurs. Comme la normalisation retire la ponctuation, c'est aussi ce contrôle qui distingue « 94.9 % » de « 949 % ».

Une citation absente du passage indiqué mais présente ailleurs dans le même article est signalée avec l'identifiant où elle a été trouvée, et la commande sort en erreur : c'est la source qui était fausse.

```
$ bin/pdb verify 1612.07119#019.3 "P-wall using a wall-power meter for the whole board power consumption"
APPROX  0.95  1612.07119#019.3
```

Enfin, `pdb paper` donne l'identité d'un article (auteurs, venue, preprint ou non, statut de la fiche), et `pdb cites` / `cited-by` parcourent le graphe des citations internes au corpus, reconstruit en rapprochant les références par identifiant arXiv, DOI ou titre normalisé.

4.5 Protocole de réponse : la skill `/paper-ask`

La skill `/paper-ask` fixe à l'agent un déroulé, et ne l'autorise qu'à utiliser les commandes `bin/pdb` et la lecture de fichiers :

1. avant toute recherche, poser 2 à 5 exigences, ce qu'une réponse complète doit établir (une valeur, une plateforme, une condition, une comparaison), sans aller plus loin que ce que la question demande ;
2. question chiffrée : `pdb bench` d'abord (N0), et `bench --comparable` avant de comparer deux articles ;
3. sinon, ou en complément : `pdb search --rerank` avec une ou deux formulations en anglais ; si aucun extrait ne répond, dire que la base ne traite probablement pas le sujet, au lieu de combler le manque ;
4. `pdb show` sur 2 ou 3 passages au plus, jamais sur le texte intégral ;
5. réponse avec citations `[chunk_id]`, et `pdb verify` sur chaque citation textuelle : une citation `ABSENT` est retirée, une citation `NOMBRES` est corrigée d'après le texte exact ou retirée, un identifiant faux est corrigé, et une citation `APPROX` n'est gardée que présentée comme une paraphrase ;

6. marquer chaque exigence `couvert` (passage vérifié), `partiel` (passage voisin : autre plateforme, autre batch, autre portée de puissance) ou `absent`, et en tirer le verdict : `suffisant` si toutes sont couvertes, `absent` si aucune, `partiel` sinon ;
7. structure imposée : le verdict et les exigences en tête, puis **Lu** (sourcé), **Déduit**, **Non trouvé dans la base**, avec signalement des valeurs de confiance basse, des preprints et des portées de puissance hétérogènes.

Le verdict déplace la question « la base traite-t-elle ce sujet ? » d'un score de recherche, qui ne la tranche pas (section 6.5), vers le contenu des passages lus. Un verdict `absent` porte sur la base, pas sur la littérature : la réponse le rappelle et propose `/paper-discover`. Un mode `--record` écrit la réponse en JSON, sous forme d'affirmations citées, pour l'évaluation des réponses (section 6.9).

4.6 Un moteur optionnel sur GPU et l'acquisition open access

Moteur. Une seconde chaîne de recherche a été développée en parallèle, pour une machine équipée d'un GPU (mesures faites sur une RTX 5090 de 32 Go) : `Qwen3-Embedding-4B` pour l'index dense et `Qwen3-Reranker-4B` pour le reranking, en BF16 et à révisions de modèle épinglées. Elle est aujourd'hui un *moteur optionnel de* `pdb search` : il encode exactement les mêmes passages, avec les mêmes identifiants, de sorte que `show`, `verify`, `/paper-ask` et l'évaluation des réponses ne changent pas. Le moteur CPU reste celui par défaut. `pdb gpu on` active le moteur GPU pour la machine (réglage hors des bases), télécharge les modèles et réindexe ; `--gpu` ou `--cpu` le force pour une recherche. Sans CUDA, sans les bibliothèques nécessaires ou sans vecteurs Qwen pour la base, `search` reste sur le CPU et l'écrit : une question n'échoue jamais faute de GPU. `pdb eval --rag --gpu` écrit un rapport aux mêmes clés que le moteur CPU, ce qui permet de comparer les deux moteurs avec `--compare`.

La commande `bin/pdb rag` garde ce qui reste propre à la chaîne d'origine : un index SQLite (FTS5 et vecteurs Qwen) de preuves *typées*, où figurent, à côté des résumés, sections, tableaux et figures du texte original, les fiches terminées et les lignes de `benchmarks.csv`, chacune marquée comme transcription ou note. Une fiche ou une ligne de benchmark sert à s'orienter, et l'agent doit remonter au passage original (`rag evidence ID --parent`) avant de citer. `rag benchmarks` filtre les chiffres sans plafond de résultats ; `rag check-answer` vérifie, pour une réponse enregistrée en JSON, que chaque extrait cité figure dans la preuve invoquée, sans encore contrôler les nombres comme `verify`. La skill `/paper-research` encadre cette route.

Acquisition. Quand le corpus ne suffit pas, `rag acquire` cherche des textes intégraux en accès ouvert (OpenAlex, arXiv), les télécharge dans un répertoire privé, les extrait et les déduplique par DOI, identifiant arXiv et empreinte du document. Rien n'entre dans la base à ce stade : l'agent présente chaque article retenu (titre, année, extrait) et n'appelle `rag approve` que pour ceux que l'utilisateur accepte. Un article importé reçoit une fiche `todo` et n'écrase jamais un article déjà présent. Un budget borne chaque session : 8 recherches, 8 PDF, 50 Mo par PDF, 200 Mo et 15 minutes au total. Un PDF indisponible est traité comme une limite de la recherche, pas comme une preuve d'absence.

Mesure. Les deux moteurs ont été comparés sur les jeux de questions de la section 6.4, avec les mêmes attentes et les mêmes métriques (tableau 5). Cette comparaison date du 26/09/2026, sur le corpus de 28 articles, avec la chaîne `pdb rag` d'origine : elle n'a été refaite ni sur le corpus actuel ni avec le moteur intégré à `pdb search`, faute de GPU sur la machine des mesures suivantes.

Le seul écart net est le français : Qwen retrouve les 8 questions sans reformulation en anglais. Sur le holdout, il gagne 2 questions sur 25 au recall@5, à MRR comparable. Sur le jeu dev, il fait moins bien, ce qui est en partie attendu puisque ce jeu a servi à régler le premier moteur. Son canal lexical (FTS5) est plus faible que le BM25 du premier moteur (dev 0,80 contre 0,95 ; français 0,00). Le reranker Qwen est aussi le meilleur signal « hors corpus » mesuré (section 6.5). Il a un coût : un GPU (chaque modèle occupe environ 8 Go), et des modèles rechargés à chaque

TABLE 5 – Moteur CPU (bge-small) contre moteur GPU (Qwen3), mêmes jeux de questions, corpus de 28 articles. En gras : meilleure valeur par jeu.

Jeu	Moteur	recall@5	recall@10	MRR	nDCG@10	s / question
dev (20)	bge-small hybride + reranking	**1,00**	**1,00**	**0,85**	**0,46**	∼1,4
	Qwen hybride	0,90	0,90	0,78	0,37	0,7
	Qwen hybride + reranking	0,80	0,80	0,76	0,35	3,7
holdout (25)	bge-small hybride + reranking	0,80	0,80	**0,78**	**0,41**	∼1,4
	Qwen hybride + reranking	**0,88**	**0,92**	0,75	0,35	3,3
FR (8)	bge-small dense (meilleur mode)	0,62	0,75	0,46	0,19	0,1
	Qwen dense	**1,00**	**1,00**	0,63	0,22	0,7
	Qwen hybride + reranking	**1,00**	**1,00**	**0,88**	**0,31**	3,8

requête, soit 3 à 4 s par question avec reranking. Le moteur CPU reste donc celui par défaut ; le moteur GPU est une aide pour les questions en français et les écarts de vocabulaire.

5 Ce que le logiciel apporte

Économie de tokens. Le tableau 6 compare, pour quatre tâches courantes, la lecture directe des fichiers et le passage par les commandes. Les ordres de grandeur sont relevés sur le corpus actuel (taille des fichiers divisée par quatre).

TABLE 6 – Coût en tokens de tâches courantes (ordres de grandeur).

Tâche	Sans la couche de recherche	Avec
Un chiffre sur une plateforme	lire `benchmarks.csv` en entier : ∼27 000	`bench` filtré : 100–400
Trouver où un sujet est traité	`grep` sur les sections, qui échoue sur les synonymes	`search -k 8` : ∼600
Lire la preuve d'une affirmation	ouvrir `fulltext.md` : ∼12 000 (médiane)	`show` sur 1 à 3 passages : 300–900
Vérifier une citation	relire le passage	`verify` : ∼20

Vérifiabilité. Chaque élément renvoyé se rejoue : un passage par son identifiant (`show`) et sa page dans le PDF, une citation par `verify`, un chiffre par sa ligne de `bench` et sa section source, un classement par le déterminisme de la fusion. Une absence de résultat est présentée comme un constat sur la base, pas sur la littérature. Les paraphrases, les rapprochements entre articles et les inférences ne se vérifient pas automatiquement : l'agent doit les étiqueter comme déduits.

Au-delà de la recherche. La même base sert à d'autres tâches de la préparation de thèse :

- les références extraites alimentent `external_refs.csv`, qui classe les travaux absents de la base par nombre d'articles du corpus qui les citent : ce sont les prochaines lectures ;
- `pdb discover` (`/paper-discover`) interroge OpenAlex (citations entrantes, recherche par axe ou par lacune) et propose des candidats, ajoutés seulement avec l'accord de l'utilisateur ;
- `/paper-gaps` tient une carte des zones non couvertes (`gaps.md`), chaque entrée étant appuyée par des identifiants d'articles ;
- des états de l'art en LaTeX ont été rédigés par l'agent à partir des fiches des bases fpga et drone (sections par axe, glossaires, liens vers les articles), puis relus ; chaque affirmation y renvoie à un article de la base ;
- une application de visualisation locale (graphe 3D du corpus, frise des citations) donne une vue d'ensemble à l'utilisateur ; l'agent ne s'en sert pas ;

— `pdb plot` produit des figures statiques (PNG, PDF ou SVG) à partir de `benchmarks.csv`, du dernier rapport de `pdb eval` et des métadonnées du corpus : débit contre puissance, GOPS/W par précision, ressources FPGA, précision contre débit, qualité de la recherche, années, axes, citations, lacunes. Chaque figure indique en pied sa source et son effectif (par exemple 82 lignes sur 299, les `n/r` étant exclues), et `figures/manifest.json` les recense. La figure 2 en est un exemple ; les portées de puissance hétérogènes y sont distinguées plutôt que mélangées.

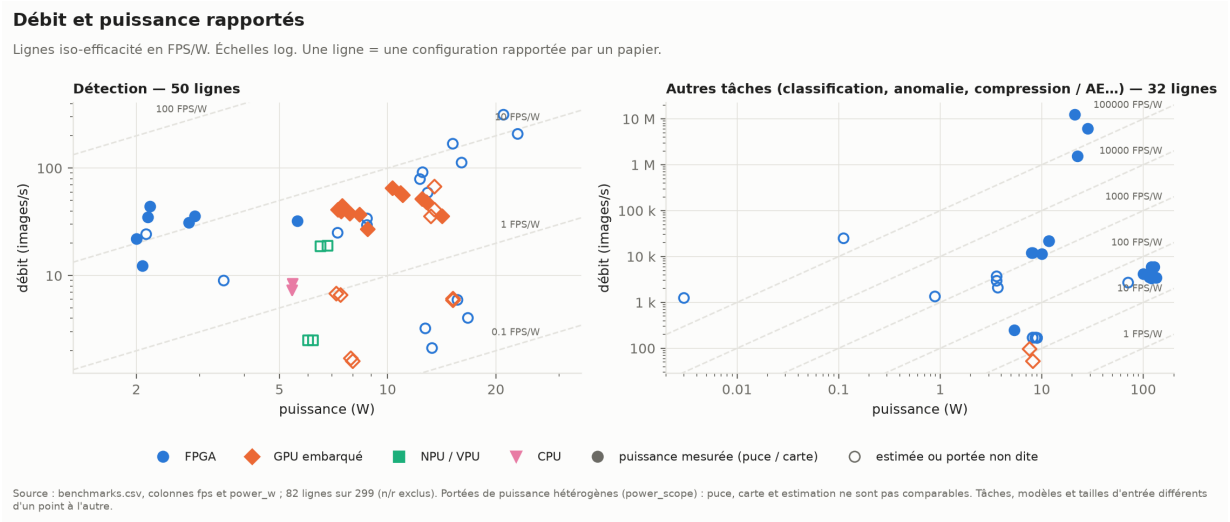


FIGURE 2 – Sortie de `pdb plot bench` sur la base fpga : débit et puissance rapportés, une configuration par point (82 lignes de `benchmarks.csv` sur 299), la détection à gauche, les autres tâches à droite. Symboles pleins : puissance mesurée ; creux : estimée ou portée non précisée. Les points ne sont pas comparables entre eux sans lire la portée de la mesure (section 3.2, `bench --comparable`).

Déléguer à des sous-agents. L'agent principal peut confier une tâche à un sous-agent qui démarre sans son contexte : une fiche par agent, un article à extraire, une recherche de DOI. Les règles du dépôt fixent le modèle selon la tâche : le plus léger pour la mécanique (ajout, réindexation, contrôle des métadonnées), un modèle intermédiaire pour la lecture et l'extraction, le plus capable pour la synthèse et le jugement. Deux agents n'écrivent jamais le même fichier, et `benchmarks.csv`, `INDEX.md` et la réindexation restent à l'agent principal, qui vérifie ce qu'on lui rapporte. Pour savoir si cela paie, un hook `SubagentStop` de Claude Code journalise chaque sous-agent (tokens, coût, durée, outils) dans `.cache/agents.jsonl`, et `pdb agents --report` en fait le bilan. Le coût tiré des transcriptions sous-estimait les tokens de sortie ; il est calé sur le coût que Claude Code compte pour la session. Sur 72 sous-agents journalisés (126 M tokens, 81 $), la délégation aurait fait économiser environ 59 % du coût d'un travail fait en direct. Ce chiffre est une *estimation* : mêmes tours au tarif de l'agent principal, plus la relecture du travail à chaque tour suivant.

Pour la caler, une même tâche (trois fiches de lecture de détecteurs YOLO de la base drone) a été faite deux fois dans des sessions neuves, en direct puis déléguée à trois sous-agents en parallèle (tableau 7). Le modèle prédit 2,21 $ pour la session directe, qui a coûté 2,30 $ (écart de 4 %). La délégation a coûté 41 % de moins et pris 59 % de temps en moins. L'essai est unique, et les deux sessions n'ont pas produit le même travail : en direct, les fiches sont plus longues et comptent 9 lignes de benchmark de plus. Une part de l'écart vient donc du travail, pas seulement du tarif, et la qualité n'a pas été notée. L'expérience a aussi fait apparaître un défaut du journal : le hook passait avant l'écriture du dernier tour du sous-agent, qui manquait au décompte ; il relit désormais la transcription jusqu'à ce qu'elle soit complète.

TABLE 7 – Expérience appariée : trois fiches de lecture, en direct et déléguées (03/10/2026, un seul essai).

	Direct	Délégué
Coût compté par Claude Code	2,30 $	1,35 $ (0,46 + 0,89 pour les sous-agents)
Durée	324 s	133 s
Tours de l'agent principal	29	6 (+ 24 pour les 3 sous-agents)
Contexte principal en fin de session	139k tokens	54k tokens
Lignes de `benchmarks.csv` / taille des fiches	24 / 23,5k car.	15 / 17,8k car.

État du corpus. Au 3 octobre 2026, la base fpga compte 101 articles, tous avec une fiche terminée, 299 lignes de benchmark, 8 601 passages indexés et 37 lacunes recensées, dont 29 appuyées par des articles du corpus (sortie de `pdb status`). Le corpus a plus que doublé depuis la version précédente de ce texte (45 articles, 3 479 passages), en grande partie avec des articles sur la sûreté et la certification de l'IA embarquée ; les fiches et lignes de benchmark des nouveaux articles ont été rédigées par des sous-agents, puis vérifiées par l'agent principal.

6 Tests et évaluation

6.1 Tests unitaires

La suite (`bin/pdb test`) compte 286 tests répartis en 26 fichiers. Elle s'exécute en moins de 30 secondes, sans GROBID et sans modèle d'embeddings, grâce à des substituts déterministes (tableau 8) :

- une vraie sortie TEI de GROBID (0.8.1), conservée comme fixture, pour le parsing ;
- un encodeur et un cross-encoder factices, fonctions déterministes du texte, pour tester la fusion, le reranking et l'index incrémental sans sentence-transformers ;
- des réponses OpenAlex simulées pour la découverte et l'ajout par DOI ;
- un PDF synthétique produit par PyMuPDF (une légende suivie d'un tableau à filets) pour la reconstruction des tableaux, et des grilles construites à la main pour l'appariement des tableaux Docling ;
- de petits corpus synthétiques (dont une chaîne de citations datée) pour le graphe et la généalogie ;
- des bases et un registre temporaires pour la résolution de la base active, la création d'une base, le scan et l'import entre bases ;
- des transcriptions de sous-agents construites à la main, pour le journal et le calcul du coût de la délégation.

Les tests de la visualisation et des figures sont ignorés si leurs dépendances optionnelles ne sont pas installées. Le dernier run (03/10/2026) réussit les 286 tests, y compris celui de l'acquisition open access, qui échouait au run précédent faute de pouvoir charger le tokenizer Qwen sans réseau.

Deux garde-fous complètent ces tests à l'usage : `reindex` valide chaque fichier contre le schéma et s'arrête au premier écart, et une liste de contrôles manuels de la chaîne d'ingestion (métadonnées, ordre des sections, absence d'en-têtes résiduels, repli sans GROBID, recoupement des chiffres d'une fiche avec le PDF) est tenue dans la documentation technique.

6.2 Le banc d'évaluation `pdb eval`

Les tests unitaires vérifient que le code fait ce qu'il annonce. Ils ne disent pas si l'extraction est bonne ni si la recherche trouve les bons passages. `bin/pdb eval` le mesure sur le corpus réel, sans LLM ni réseau, et sans rien écrire dans la base. Il couvre quatre volets : l'extraction, la recherche, la vérification des citations et les temps ; un cinquième, les réponses rédigées, est décrit en section 6.9. Un run complet prend quelques minutes sur CPU. `--save` enregistre le

rapport en JSON (daté, avec le commit mesuré), et `--compare` signale les écarts avec le dernier rapport, avec un code d'erreur si un seuil de `eval/thresholds.json` n'est plus atteint. La démarche est la suivante : enregistrer avant un changement (extraction, découpage, modèle, fusion), le faire, puis comparer. Le banc porte sur la base active : une base peut avoir ses propres jeux de questions et ses seuils, et ses rapports sont enregistrés dans son dossier (section 6.10). Les chiffres ci-dessous portent sur la base fpga et proviennent d'un run du 03/10/2026 sur 101 articles et 8 601 passages (`bases/fpga/eval/results/2026-10-03T2340-3925690.json`). Ils sont comparés, entre parenthèses, au run de référence du 28/09 sur 45 articles et 3 479 passages (`2026-09-28T1039-76dc85e`) : les jeux de questions n'ont pas changé, mais le corpus a plus que doublé et le code a évolué (découpage des tableaux, moteur de recherche restructuré). Un écart entre les deux runs mêle donc ces deux causes.

6.3 Qualité de l'extraction

Le banc calcule d'abord des métriques intrinsèques, qui ne demandent aucune annotation. Il réutilise ensuite deux vérités terrain déjà présentes dans la base : les métadonnées vérifiées à la main et les valeurs relevées dans `benchmarks.csv` (tableau 9).

- *La couverture* ne peut pas atteindre 1 : en-têtes, affiliations, texte des figures vectorielles et mots coupés par une césure sont exclus volontairement. Elle se lit en relatif, d'un run ou d'un article à l'autre. Le minimum (0,57) est une thèse de 103 pages ; les deux suivants sont des documents réglementaires de l'agence européenne de la sécurité aérienne (EASA), entrés avec l'axe sûreté et certification.
- *L'année et le DOI* sont bas surtout par construction : le PDF arXiv ne porte souvent ni date ni DOI de la version publiée. C'est pour cela que l'ingestion les complète depuis arXiv et depuis les exports de citation. Le banc a aussi révélé un vrai défaut, encore ouvert : un DOI gabarit d'éditeur (MDPI) n'est pas rejeté.
- *Tableaux.* La part des tableaux avec corps baisse (0,88 → 0,78), et la baisse vient entièrement des 56 nouveaux articles : les 45 anciens gardent 25 tableaux sans corps sur 233, les nouveaux en ont 103 sur 351, dont 29 dans les deux documents de l'EASA. Docling avait fait passer cette part de 0,66 à 0,90 sur 28 articles, avec OCR pour les tableaux publiés en image ; mais la qualité des tableaux passés par l'OCR est inégale (un contrôle à l'œil de trois tableaux en trouve un propre, un incomplet et un en partie illisible), d'où la mention « à vérifier ». Les signaux de qualité (section 3) signalent 255 tableaux sur 584 : 163 au nombre de cellules irrégulier, 107 aux nombres collés, 82 aux cellules vides, 14 aux caractères d'OCR douteux. Les échantillons relus de ces deux dernières catégories sont de vrais défauts (trois valeurs dans une cellule, premières lettres perdues). Le taux « avec corps » mesure la présence d'un corps, pas l'exactitude des cellules : une vérité terrain par tableau reste à construire.
- *Les valeurs retrouvées remontent* (0,65 → 0,70 dans le tableau cité) après une baisse au passage de 28 à 45 articles. Les lignes de benchmark des nouveaux articles sont donc, en moyenne, mieux ancrées dans leur tableau. Le banc ne dit pas si une valeur manquante vient de l'extraction ou de la transcription faite par l'agent ; c'est ce que `bench --check` contrôlera ligne par ligne, quand les lignes porteront leur passage (`chunk_id`) : aucune ne le fait encore.

6.4 Recherche de passages

Jeux de questions. Chaque question est associée aux sections pertinentes du corpus (identifiées par `article#ordre`, indépendamment du découpage en fenêtres). Ces sections sont choisies par motif textuel dans les articles cibles (par exemple *reuse factor* dans les deux articles hls4ml). Les questions reformulent le sujet sans reprendre le motif quand c'est possible : « removes duplicate bounding boxes » pour la non-maximum suppression. Il y a trois jeux :

- *dev* : 20 questions en anglais, qui ont servi aux choix de conception (modèle, fusion, reranking) ;

— *holdout* : 25 questions en anglais, écrites ensuite et jamais utilisées pour régler quoi que ce soit. Elles couvraient les 12 articles absents du jeu dev et comprennent 2 questions sur des tableaux et 3 questions qui portent sur plusieurs articles. Trois attentes ont été élargies après lecture des passages renvoyés, parce que ces passages répondaient bien à la question (avant cela : recall@5 de 0,68 avec reranking) ;
— *FR* : 8 questions du jeu dev traduites à la main en français.

Ces jeux ont été écrits pour un corpus de 28 à 45 articles. Sur les 101 actuels, environ 60 % des articles ne sont visés par aucune question.

Métriques. recall@5 et recall@10 (au moins une section attendue parmi les 5 ou 10 premiers passages), MRR (inverse du rang de la première bonne section), nDCG@10 à gain binaire (Järvelin and Kekäläinen, 2002), qui récompense le fait de trouver *plusieurs* sections pertinentes, et paper@5 (au moins un passage d'un article cible parmi les 5 premiers).

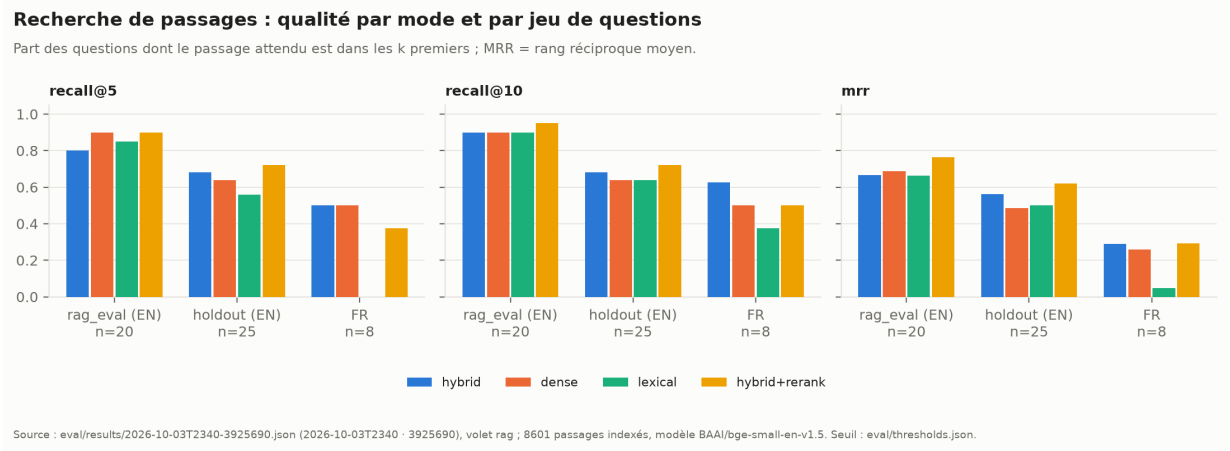


FIGURE 3 – Les chiffres du tableau 10, tracés par `pdb plot eval` à partir du rapport JSON du banc. La ligne pointillée est le seuil d'acceptation (recall@5 $\geq$ 0,8).

Lecture des résultats.

— *Le holdout passe sous le seuil.* Avec 101 articles, plus aucun mode n'atteint un recall@5 de 0,8 sur le holdout : le meilleur, l'hybride avec reranking, tombe de 0,80 à 0,72, soit deux questions de moins. Le jeu dev perd aussi (1,00 $\rightarrow$ 0,90), comme le français (0,50 $\rightarrow$ 0,38). C'est le résultat le plus important de ce run, et sa cause n'est pas établie : le corpus a plus que doublé, ce qui ajoute des passages voisins des passages attendus ; les tableaux sont désormais indexés par rangées ; et le code de recherche a été restructuré. Le seul changement mesuré isolément, le découpage des tableaux, ne modifie ni le dev ni le holdout. Le diagnostic prévu rejoue les jeux sur l'index restreint aux 45 articles du run de référence, puis l'ancien code sur le corpus actuel, pour séparer l'effet du corpus de celui du code. Le seuil n'est pas abaissé, et rien n'est réglé sur le holdout.
— *Le jeu dev était optimiste.* Sur le holdout, le recall@5 perd 12 à 29 points selon le mode, et le lexical est le plus faible. Cela confirme le biais attendu : sur le jeu dev, les sections attendues avaient été choisies par mots-clés, ce qui avantageait BM25.
— *Le reranking reste le meilleur mode en anglais*, sur les deux jeux. Il ne compense plus, à lui seul, la concurrence des nouveaux articles.
— *Échecs réels du holdout* (avec reranking) : la tolérance aux radiations (deux questions), l'entraînement quantifié, la mémoire d'exemples passés, la distillation de jeu de données, et les deux questions sur des tableaux (ressources des autoencodeurs d'un article hls4ml, prototypes de FINN). Pour ces deux dernières, le constat initial (« le texte d'un tableau se réduit à sa légende ») ne tient plus : depuis Docling, ces tableaux ont un corps et sont assez

courts pour ne pas être découpés. L'échec vient du vocabulaire : la question dit « resource utilization », « throughput », « autoencoder » ; le tableau dit « LUT », « Thr.put », « AE ». Préfixer les passages de tableau du titre de l'article a été essayé, et n'aide pas.

— *En français, le lexical s'effondre* (recall@5 de 0,00), et l'index dense résiste mieux, alors que le modèle n'est pas conçu comme multilingue. Le reranking, entraîné sur MS MARCO en anglais, dégrade le classement. Le moteur GPU (section 4.6) faisait nettement mieux sur ce jeu.

— *Décisions.* L'hybride reste le mode par défaut de `pdb search`, parce que le reranking dégrade le français, selon une règle fixée avant la mesure. `/paper-ask` l'active, parce qu'il formule toujours ses requêtes en anglais et que le rang de la première bonne réponse compte quand on n'ouvre que 2 ou 3 passages.

Lors d'une mesure antérieure (25/09/2026, jeu dev), le modèle `all-MiniLM-L6-v2` obtenait un recall@5 de 0,75 en mode dense contre 0,90 pour bge-small, ce qui a motivé le choix de ce dernier. Le seuil d'acceptation retenu est un recall@5 d'au moins 0,8 en anglais.

6.5 Détecter qu'une question sort du corpus

Peut-on dire, sans LLM, que la base ne traite pas une question ? Deux jeux de 10 questions hors corpus (repliement des protéines, blockchain, etc.) ont été écrits : l'un sert à la calibration, l'autre à la validation. Le second a été écrit avant toute mesure du cross-encoder. Deux signaux sont mesurés : le meilleur cosinus de la recherche dense, et le meilleur score du cross-encoder sur les 3 premiers passages. Pour chacun, le seuil est choisi sur la calibration (jeu dev et premier jeu hors corpus, dont une question à moitié couverte par le corpus a été écartée), en maximisant l'exactitude équilibrée, puis appliqué tel quel à la validation (holdout et second jeu hors corpus). La règle d'adoption a été fixée avant la mesure : en validation, au moins 80 % des questions hors corpus signalées, et au plus 5 % des questions du corpus.

Aucun signal ne remplit la règle. Avec bge-small, une question hors sujet obtient un cosinus aussi élevé qu'une question pertinente : le calibrage initial, fait sur quelques requêtes, ne tenait pas, et l'alerte ne se déclenchait jamais. Le cross-encoder sépare mieux les médianes, mais certaines questions du corpus tombent très bas, souvent celles où la recherche échoue déjà. L'alerte a donc été retirée de `pdb search`. Le reranker du moteur GPU est le signal le plus proche de la règle : il signale les 10 questions hors corpus, mais aussi 4 des 25 questions du holdout (16 %, pour 5 % tolérés). Il n'est pas adopté non plus. La réponse est venue d'ailleurs : quand c'est l'agent qui juge, sur le contenu des passages lus, il reconnaît les questions hors corpus (section 6.9).

6.6 Vérification des citations

Le banc tire 40 passages au hasard (graine fixe) et fabrique quatre sortes de citations : une phrase exacte, qui doit être acceptée ; la même phrase avec un mot remplacé, puis avec un chiffre modifié, et une phrase d'un autre article, qui doivent être refusées. « Acceptée » signifie un code de sortie nul (`OK` ou `APPROX` dans le passage indiqué).

Avant le contrôle, `verify` acceptait toutes les citations dont un chiffre avait été modifié, alors que les règles de travail de l'agent comptent sur lui pour garantir les nombres cités. Une première version du contrôle en laissait encore passer 6 (un « 4 » reconnu dans « 6.4 », un « 2 » trouvé ailleurs dans le passage). L'exigence des mots voisins règle ces deux cas, mais pas tous. Sur la base drone (section 6.10), le run du 03/10 accepte, pour la première fois, une citation au chiffre modifié : « before step 2 » au lieu de « before step 4 », dans un passage qui contient ailleurs « in step 2 and 3 ». Le nombre modifié et son mot voisin existent bien dans le passage, mais pas à cet endroit. Ce cas n'est pas corrigé ; il montre que le contrôle garantit la présence de chaque nombre avec son contexte immédiat, pas sa position dans la phrase. Reste aussi non traité : un *mot* remplacé loin de tout nombre donne toujours `APPROX`. C'est pourquoi `/paper-ask` ne garde une citation `APPROX` que présentée comme une paraphrase.

6.7 Temps d'exécution

Le coût d'une recherche vient presque entièrement du démarrage (import de PyTorch, chargement du modèle) : 6,6 s à froid contre 0,1 s à chaud. C'est le coût réel d'un appel par l'agent, qui lance une commande à chaque fois. Avec 2,5 fois plus de passages qu'au run du 28/09, les temps de recherche à chaud n'ont presque pas bougé, et le chargement de l'index passe de 0,29 à 0,52 s. Le temps de `verify`, qui était passé de 4 à 246 ms sans explication, est revenu à 2 ms.

6.8 Essai réel de bout en bout

Question posée à `/paper-ask`, sur le corpus de 28 articles : « comment les papiers mesurent-ils la puissance, et est-ce comparable ? ». L'agent a enchaîné une requête `bench` (57 lignes, 8 articles), une recherche de 10 passages, la lecture de 3 d'entre eux, 7 vérifications de citations et 8 requêtes d'identité. L'ensemble représente environ 9 Ko lus, soit environ 2 500 tokens, sans ouvrir de texte intégral. Les 7 citations textuelles sont revenues `OK` au premier essai. La recherche a retrouvé en une requête les quatre protocoles de mesure du corpus (PMBus et prise murale, multimètre à l'entrée de la carte, `tegrastats`, puissance au repos mesurée avant les tests). La structure Lu / Déduit / Non trouvé a fait ressortir deux points à vérifier dans la base elle-même, dont un article qui décrit une mesure soignée sans qu'aucune ligne de puissance n'ait été saisie. L'essai a aussi révélé deux défauts, corrigés depuis et couverts par un test chacun : un identifiant court tronqué pour un nom d'auteur composé, et une agrégation qui demandait un script externe (d'où `bench --count`). Cet essai était unique et jugé par l'agent qui l'avait exécuté ; la section suivante le remplace par une évaluation répétable.

6.9 Évaluer la réponse rédigée

Les volets précédents mesurent le classement des passages. Ils ne disent pas si la réponse finale reconnaît une question hors corpus, si ses citations tiennent, ni si elle retrouve les sources et les valeurs attendues. `pdb eval --answers` le mesure, toujours sans LLM : les réponses sont produites à part, puis notées.

Protocole. Chaque question d'un jeu de réponses porte une catégorie, les exigences d'une réponse complète, les sources attendues (sections ou articles), les valeurs attendues avec leur unité, et le verdict attendu. Il y a quatre catégories : un fait sourcé dans un article (*simple*), des valeurs avec leurs conditions (*numérique*), une synthèse sur plusieurs articles (*synthèse*), et une question proche du thème mais que le corpus ne couvre pas (*absent* : polypes en endoscopie, langue des signes, sonar, EEG…, vérifiés absents par recherche plein texte). La skill `/eval-answers` confie chaque question à un sous-agent qui ne reçoit que son texte, ni catégorie, ni sources, ni verdict attendu, et qui répond par `/paper-ask --record`. La réponse enregistrée est une liste d'affirmations marquées lues ou déduites, chacune avec ses citations et leurs extraits. Le banc calcule l'exactitude du verdict, la part des citations acceptées (`verify` renvoie `OK` ou `APPROX` dans le passage indiqué, ou la section citée figure parmi les sources des lignes de `bench` de l'article), la part des affirmations « lues » sans aucune citation acceptée, et le rappel des sources et des valeurs attendues. Une grille de notes humaines (exigences, soutien, conditions, limites, de 0 à 2) est prévue : les contrôles automatiques vérifient que les extraits existent, pas qu'ils soutiennent la phrase.

Premiers runs. Les jeux de départ de la base fpga comptent 12 questions de développement et 12 réservées, 3 par catégorie, écrites par l'auteur du code. Deux runs ont été faits le 02/10/2026, une réponse par question, par des sous-agents Sonnet ; le second avec la version de `/paper-ask` qui pose les exigences et le verdict de suffisance (section 4.5). Les deux runs ont été faits dans une configuration dégradée : les modèles de la recherche dense et du reranking n'étaient pas téléchargeables depuis le conteneur, et la recherche était donc lexicale seule (tableau 14).

— *Hors corpus.* Aux deux runs, les 6 questions *absent* sont reconnues comme telles, et aucune question du corpus n'est déclarée absente à tort. Les réponses disent que la base ne traite pas le sujet, sans le combler par des connaissances générales. C'est ce qu'aucun score de recherche ne permettait d'obtenir (section 6.5). Avec 6 questions, c'est un signal, pas une mesure.
— *Citations.* Au second run, toutes les citations sont acceptées et toute affirmation « lue » en porte au moins une.
— *Synthèse.* C'est la catégorie faible : 3 verdicts justes sur 6 au premier run, 2 au second. L'agent répond `partiel` là où le jeu attend `suffisant`, parce qu'une exigence de la question (budget mémoire de chaque méthode, comparabilité des mesures) n'est couverte qu'en partie par le corpus. Ces réponses sont défendables ; la frontière entre suffisant et partiel est à préciser dans le jeu plutôt que dans la skill. La baisse du verdict juste au second run vient de là, et d'une réponse où la règle nouvelle (un verdict `suffisant` exige que toutes les exigences soient couvertes) corrige une incohérence du premier run.
— *Variance.* 22 verdicts sur 24 sont identiques d'un run à l'autre : le banc est assez stable pour comparer des versions de la skill sur les verdicts. Il ne l'est pas encore sur le rappel des sources (0,67 → 0,50 sur le holdout sans changement de la recherche). Ce rappel sous-estime d'ailleurs les réponses : la plupart des écarts viennent d'une source équivalente (la ligne de `bench` ou le passage du tableau plutôt que la section de texte attendue).

Aucun seuil n'est encore fixé : il manque un run en recherche hybride avec reranking, des jeux plus grands (40 questions de développement et 60 réservées par base, écrites ou relues par un tiers) et les notes humaines.

6.10 Un second corpus : la base drone

La base drone (section 3.3) a ses propres jeux de questions, écrits selon le même protocole sur ses 73 premiers articles : 20 questions de développement, 20 réservées, 8 en français et deux jeux de 10 questions hors corpus. Les seuils sont ceux de la base fpga, fixés avant le premier run. Le tableau 15 compare les deux bases (runs du 03/10/2026 au commit 3925690).

La lecture demande de la prudence :

— **Le jeu réservé drone tient son seuil**, mais l'on ne peut pas en conclure que la recherche y marche mieux. Le corpus drone est plus hétérogène (détection, suivi, neuromorphique, course de drones) : un fait distinctif y a moins de voisins proches que dans un corpus centré sur les accélérateurs. Deux attentes ont en outre été élargies après lecture des passages renvoyés. Sur 20 questions, 0,05 correspond à une question. Entre le 28/09 et le 03/10, avec 14 articles et 836 passages de plus, l'hybride perd une question sur chaque jeu anglais, et le reranking n'en perd aucune.
— **Les échecs sont de même nature** que sur la base fpga : un tableau (puissance au repos de plusieurs cartes) manqué par tous les modes, et la moitié des questions françaises.
— **La pertinence faible reste inutilisable** : le cosinus signale toutes les questions hors corpus de validation, mais aussi 2 des 20 questions réservées, au-delà des 5 % de fausses alertes tolérés.
— **Les tableaux.** Le passage de Docling sur cette base (29/09) a fait monter la part des tableaux avec corps de 0,71 à 0,90 et les valeurs retrouvées dans leur tableau de 3 points, à corpus constant : c'est le premier gain de Docling mesuré sur ces valeurs. La part retombe à 0,79 sur 87 articles parce que les 14 articles ajoutés depuis l'ont été sans leur PDF : extraits par PyMuPDF sans GROBID, ils n'ont aucun tableau avec corps (0 sur 50) et ne peuvent pas passer par Docling.
— `verify` accepte une citation au chiffre modifié sur 30, et le seuil correspondant n'est plus atteint (section 6.6).

La base stage. Elle n'a pas encore de jeux de questions : le banc ne mesure que son extraction (56 articles, 3 640 passages, run `bases/stage/eval/results/2026-10-03T2344-3925690.json`). La couverture moyenne est de 0,91 (minimum 0,61 pour le reprint numérisé de Mahalanobis, 1936), mais seuls 71 % des 254 tableaux ont un corps, aucun n'ayant été repris par Docling, et seules 42 % des 112 valeurs de `benchmarks.csv` sont retrouvées dans le tableau cité (57 % n'importe où dans le texte) : ces lignes sont les premières à relire. C'est aussi la seule base où restent des ligatures non réparées (0,37 pour 10 000 mots). Sans jeux propres, le banc retombait jusqu'ici en silence sur les questions de la base fpga, dont le rappel n'a aucun sens ici : ce repli doit devenir un avertissement.

6.11 Banc synthétique (`pdb synth`)

Les jeux précédents sont petits et écrits par l'auteur du système. `bin/pdb synth` prépare une évaluation plus large, séparée de la base de travail (`.cache/bench/`) :

- *corpus gelé* de 200 articles, avec des sujets voisins servant de distracteurs ; les passages d'ancrage des questions sont filtrés sur le FPGA, l'IA embarquée et l'accélération matérielle, en excluant introductions et généralités ;
- *génération locale* des questions par un modèle ouvert (Qwen3.8-27B quantifié en Q6, servi par llama.cpp sur la même RTX 5090), en JSON contraint par un schéma, avec graine fixe ; l'identité du modèle servi et chaque citation sont contrôlées ;
- *validation sans annotation humaine* : deux sessions indépendantes d'un LLM (Codex pour les 200 questions de référence, puis Claude Code en mode non interactif, `claude -p`) contrôlent chaque question ;
- *quotas* : 40 questions à un passage, 40 numériques, 50 inter-articles, 30 intra-article, 20 sur des tableaux et 20 sans réponse dans le corpus ; partage 40 % développement, 60 % test, sur des ensembles d'articles disjoints, équilibré entre anglais et français, et 40 questions déclinées en variantes traduites et abrégées ;
- *ablations* : les quatre modes de recherche du moteur GPU et les deux modes CPU de `pdb search` sur les mêmes questions, puis des variantes d'agent exécutées dans un bac à sable (bubblewrap, outils de lecture seuls, corpus en lecture seule), notées par un juge LLM.

Les fiches et transcriptions numériques produites automatiquement forment un corpus *enrichi* distinct, qui ne sert jamais à écrire les réponses de référence ; une ligne de benchmark n'y est gardée que si chacun de ses nombres figure dans la section citée. Le protocole et ses tests sont en place ; ses résultats ne sont pas encore disponibles et ne sont donc pas rapportés ici.

6.12 Menaces à la validité

- Les jeux sont petits : sur 25 questions, un écart de 0,04 correspond à une seule question, et aucun écart n'est lisible en dessous de deux questions. Le holdout n'a servi à régler aucun paramètre, mais il a été écrit, comme le jeu dev, par l'auteur du système. Des questions écrites par un tiers sont nécessaires pour confirmer les écarts. Le second corpus (section 6.10) montre que les tendances ne tiennent pas à un seul corpus, mais ses questions ont été écrites par le même auteur.
- Les jeux n'ont pas suivi la croissance des bases : écrits pour 45 et 73 articles, ils interrogent aujourd'hui moins de la moitié du corpus fpga, et la base stage n'en a aucun.
- Les sections attendues sont choisies par motif textuel. C'est une approximation : une bonne réponse formulée autrement, ou trouvée dans un article ajouté après l'écriture du jeu, est comptée comme un échec.
- La vérité terrain des tableaux ne couvre que les valeurs reprises dans `benchmarks.csv`, et le taux de tableaux « avec corps » ne dit rien de l'exactitude des cellules.
- Les réponses rédigées sont notées automatiquement sur 24 questions, en recherche lexicale seule, et les notes humaines n'ont pas encore été attribuées : la validité des citations est contrôlée, pas le fait qu'elles soutiennent la phrase.

— La comparaison des deux moteurs porte sur le corpus de 28 articles et sur la chaîne `pdb rag` d'origine ; elle n'a pas été refaite avec le moteur intégré à `pdb search`, ni sur le corpus actuel.
— L'économie de la délégation repose sur une estimation, calée par un seul essai apparié, sur une seule tâche, sans notation de la qualité.
— Le banc synthétique aura ses propres biais : questions générées par un LLM, validées et notées par d'autres LLM. Il mesurera ce corpus gelé et cette distribution de tâches, pas le travail de recherche en général.

7 Limites et perspectives

— **Passage à l'échelle.** De 45 à 101 articles, le recall@5 du holdout avec reranking passe de 0,80 à 0,72, sous le seuil. Avant tout réglage, il faut séparer l'effet du corpus de celui du code, en rejouant les jeux sur le sous-corpus commun et l'ancien code sur le corpus actuel. Le plan suivant est spécifié : des jeux propres à chaque base, sans repli silencieux ; une taille proportionnelle au corpus (holdout d'au moins 40 questions ou la moitié du nombre d'articles) ; des quotas par axe de la base et par type de question (tableaux, plusieurs articles) ; des questions proposées par la chaîne du banc synthétique et relues par un humain ; un nouveau run de référence à chaque croissance de 25 % du corpus ; et des intervalles de confiance par bootstrap, pour ne signaler un écart que s'il en sort.
— **Tableaux.** Ils restent la partie la plus fragile de l'extraction : 22 % des tableaux de la base fpga n'ont pas de corps, environ 40 % de ceux qui en ont un sont signalés par un indicateur de qualité, et les tableaux passés par l'OCR sont de qualité inégale. Les tableaux se retrouvent mal par la recherche, maintenant pour une raison de vocabulaire (abréviations des en-têtes) plus que de découpage ; un glossaire d'abréviations par base ou des requêtes multiples sont les pistes. Restent à faire : une skill de relecture des tableaux signalés, en commençant par ceux que cite `benchmarks.csv`, une vérité terrain d'une vingtaine de tableaux annotés à la main, et une OCR ciblée sur la zone du tableau plutôt que sur tout le PDF. Une ré-extraction (`ingest --reparse`) défait aussi certaines réparations de ligatures : à corriger avant toute ré-extraction en masse.
— **Comparaisons.** `bench --comparable` dit quelles lignes se comparent sur un axe. Une commande `pdb compare`, spécifiée, comparera sur deux axes (débit contre puissance, à métrique et jeu égaux), donnera un niveau de confiance par groupe et expliquera chaque ligne exclue. Les colonnes optionnelles restent à remplir : aucune ligne ne porte encore son passage.
— **Preuves atomiques.** Un seul article a ses `claims.jsonl`. Il reste à les produire pour les autres, à les indexer comme renvois vers leur passage, et à les faire lire par `/paper-ask`, qui devrait y gagner sur les questions de synthèse.
— **Réponses.** Il manque un run des réponses en recherche hybride avec reranking, des jeux plus grands relus par un tiers, des jeux pour les bases drone et stage, et les notes humaines.
— **Questions hors corpus.** Aucun score de recherche ne dit de façon fiable que la base ne traite pas le sujet ; le verdict de suffisance de l'agent y parvient sur les 6 questions testées, ce qui reste à confirmer sur un jeu plus grand.
— **Vérification.** `verify` refuse les chiffres modifiés quand le nombre et ses voisins ne se retrouvent pas ailleurs dans le passage, mais pas dans le cas contraire (section 6.6), et un mot remplacé loin de tout nombre passe encore pour une paraphrase (`APPROX`). Le contrôle de `rag check-answer`, sur la route GPU, ne vérifie pas encore les nombres.
— **Métadonnées.** Un DOI gabarit d'éditeur (MDPI) n'est pas encore rejeté à l'extraction ; il est corrigé à la main dans la base, mais un nouvel article du même éditeur en hériterait.
— **Langue.** Corpus et modèle par défaut sont en anglais, et la recherche sur CPU perd beaucoup en français (recall@5 de 0,38 avec reranking). Le moteur GPU retrouvait les 8 questions françaises, au prix d'un GPU. Pistes spécifiées : traduire systématiquement la

requête et fusionner les résultats, évaluer un modèle multilingue à côté du modèle actuel, et porter le jeu français à 30 questions par base.

— **Provenance.** Rien ne garde trace de l'état du corpus sur lequel une réponse a été rédigée : après une réindexation, on ne peut pas savoir sur quoi elle reposait. Une empreinte du corpus et un journal des réponses sont prévus.

— **Latence.** Environ 7 s par `search` à froid, contre 0,1 s à chaud, dues pour l'essentiel à l'import de PyTorch et au chargement du modèle ; `show` et `verify`, qui s'en passent, répondent en moins d'une seconde. Un processus résident, par exemple un serveur MCP exposant les mêmes commandes, ne paierait ce coût qu'une fois, et permettrait d'utiliser la base depuis d'autres applications. Le même argument vaut pour le moteur GPU, qui recharge ses modèles à chaque requête (3 à 4 s).

— **Généricité.** Le même code sert trois bases (section 3.3), dont une sans aucune mesure matérielle, et une nouvelle se crée par `pdb base new`. Les colonnes obligatoires de `benchmarks.csv` (FPS, GOPS, ressources FPGA) restent marquées par le domaine d'origine : sur 100 lignes, la base stage n'a aucune ressource FPGA et deux débits ; elle rapporte ses résultats en précision ou en erreur de prédiction. Les colonnes optionnelles et le bloc `plot` par base ont assoupli le schéma sans le rendre configurable.

8 Conclusion

PaperDatabase montre qu'un corpus de recherche de taille modeste peut être rendu exploitable par un agent LLM sans base de données ni service externe : des fichiers texte bien structurés, un tableau de chiffres sourcés et une recherche hybride déterministe suffisent. La génération reste à l'agent, mais tout ce qu'il cite peut être rejoué et vérifié par une commande. Le coût d'une réponse sourcée baisse d'un ordre de grandeur par rapport à la lecture des textes intégraux, et déléguer la lecture à des sous-agents moins coûteux réduit encore la facture, de 41 % sur le seul essai apparié.

Le banc d'évaluation a montré l'intérêt de mesurer ce que l'on croit acquis : un jeu de questions optimiste, une alerte qui ne se déclenchait jamais, une vérification qui laissait passer des chiffres faux, puis, en remesurant, une recherche qui perd du rappel quand le corpus double. Ce dernier point est le principal problème ouvert. L'évaluation des réponses rédigées a apporté le résultat inverse : là où aucun score ne savait dire qu'une question sort du corpus, l'agent le dit quand on l'oblige à confronter chaque exigence de la question aux passages lus. Le même code sert trois bases isolées, sur lesquelles le banc retrouve les mêmes forces et les mêmes faiblesses. Les prochaines étapes sont le diagnostic de la baisse du rappel et des jeux à l'échelle des bases, des questions écrites par un tiers, les notes humaines des réponses, une meilleure recherche dans les tableaux, la comparaison sur deux axes et un moteur résident qui évite de recharger les modèles à chaque requête.

Références

Christoph Auer, Maksym Lysak, Ahmed Nassar, Michele Dolfi, Nikolaos Livathinos, Panos Vagenas, Cesar Berrospi Ramis, Matteo Omenetti, et al. Docling technical report. arXiv :2408.09869, 2024. Preprint.

Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009. doi : 10.1145/1571941.1572114.

GROBID contributors. GROBID : machine learning software for extracting information from scholarly documents. `https://github.com/kermitt2/grobid`, 2008–2026. Logiciel, version 0.8.1 utilisée.

Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4) :422–446, 2002. doi : 10.1145/582415.582418.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.

Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. MS MARCO : A human generated MAchine reading COmprehension dataset. In *Workshop on Cognitive Computation (CoCo@NIPS)*, 2016.

Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. arXiv :1901.04085, 2019. Preprint.

Jason Priem, Heather Piwowar, and Richard Orr. OpenAlex : A fully-open index of scholarly works, authors, venues, institutions, and concepts. arXiv :2205.01833, 2022. Preprint.

Nils Reimers and Iryna Gurevych. Sentence-BERT : Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019. doi : 10.18653/v1/D19-1410.

Stephen E. Robertson and Hugo Zaragoza. The probabilistic relevance framework : BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4) :333–389, 2009. doi : 10.1561/1500000019.

Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM : Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.

Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-Pack : Packed resources for general Chinese embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024. doi : 10.1145/3626772.3657878. Preprint arXiv :2309.07597 (2023).

A Glossaire des acronymes

Acronyme	Signification
API	*Application Programming Interface*, interface de programmation
BF16	*bfloat16*, format flottant sur 16 bits
BM25	*Best Matching 25*, fonction de score de la recherche lexicale
BRAM	*Block RAM*, mémoire embarquée d'un FPGA
CPU	*Central Processing Unit*, processeur généraliste
CSV / TSV	*Comma- / Tab-Separated Values*, tableau texte à séparateur virgule / tabulation
DOI	*Digital Object Identifier*, identifiant pérenne d'une publication
DSP	*Digital Signal Processing* : bloc de calcul (multiplieur-accumulateur) d'un FPGA
FPGA	*Field-Programmable Gate Array*, circuit logique reprogrammable
FPS	*Frames Per Second*, images par seconde
FTS5	*Full-Text Search* version 5, index plein texte de SQLite
GOPS	*Giga Operations Per Second*, milliards d'opérations par seconde
GPU	*Graphics Processing Unit*, processeur graphique
GROBID	*GeneRation Of BIbliographic Data*, extracteur de structure d'articles PDF
IA	intelligence artificielle
INT8	entiers sur 8 bits (quantification)
JSON	*JavaScript Object Notation*, format de données texte
LLM	*Large Language Model*, grand modèle de langage
LUT	*Look-Up Table*, cellule logique élémentaire d'un FPGA
MCU	*Microcontroller Unit*, microcontrôleur
MCP	*Model Context Protocol*, protocole d'exposition d'outils à un agent LLM
MRR	*Mean Reciprocal Rank*, moyenne de l'inverse du rang de la première bonne réponse
MS MARCO	*Microsoft MAchine Reading COmprehension*, jeu de données de recherche de passages
nDCG	*normalized Discounted Cumulative Gain*, qualité d'un classement
NMS	*Non-Maximum Suppression*, suppression des boîtes de détection en double
OCR	*Optical Character Recognition*, reconnaissance optique de caractères
PDF	*Portable Document Format*
RAG	*Retrieval-Augmented Generation*, génération augmentée par recherche
RRF	*Reciprocal Rank Fusion*, fusion de classements par rangs réciproques
SHA-1	*Secure Hash Algorithm 1*, empreinte d'un texte
TEI	*Text Encoding Initiative*, format XML de sortie de GROBID
YOLO	*You Only Look Once*, famille de détecteurs d'objets

B Lexique

Terme	Sens dans ce document
agent	LLM qui appelle des outils (lecture de fichiers, commandes) pour accomplir une tâche ; ici Claude Code
benchmark (ligne de)	une configuration rapportée par un article (modèle, plateforme, précision, débit, puissance…), avec sa section source, dans `benchmarks.csv`
chunk, passage	fragment de section indexé pour la recherche, identifié par `<article>#<section>.<k>`
cross-encoder	modèle qui note ensemble une requête et un passage ; sert au reranking
exigence	ce qu'une réponse complète doit établir (valeur, plateforme, condition…) ; statut `couvert`, `partiel` ou `absent`
embedding	vecteur qui représente le sens d'un texte ; deux textes proches ont des vecteurs proches
fiche	lecture critique d'un article (`fiche.md`) : problème, méthode, résultats, limites, pertinence
hook	commande que Claude Code lance d'elle-même à un moment donné (ici, à la fin de chaque sous-agent)
holdout	jeu de questions réservé, qui n'a servi à régler aucun paramètre
index dense / lexical	index par embeddings (sens) / par mots exacts (BM25, FTS5)
lacune (gap)	zone que le corpus ne couvre pas, consignée dans `gaps.md`
`n/r`	« non rapporté » : valeur absente de l'article, jamais estimée
portée de puissance	ce que couvre une mesure de puissance (puce, carte, estimation), colonne `power_scope`
preuve atomique (claim)	une affirmation d'un article, avec son passage source et l'extrait exact qui la porte, dans `claims.jsonl`
preprint	article diffusé sans relecture par les pairs (arXiv)
recall@k	part des questions dont une section attendue figure parmi les k premiers passages
paper@k	part des questions dont un article attendu figure parmi les k premiers passages
reranking	reclassement des premiers résultats par un modèle plus coûteux
skill	procédure écrite que l'agent suit pour une tâche (`/paper-ask`, `/paper-fiche`…)
sous-agent	agent lancé par l'agent principal pour une tâche délimitée, sans son contexte, souvent sur un modèle moins coûteux
toolflow	chaîne d'outils qui transforme un réseau de neurones en accélérateur matériel (FINN, hls4ml…)
verdict de suffisance	en tête d'une réponse : `suffisant` (toutes les exigences couvertes), `partiel` ou `absent` (aucune)

C Commandes

Commande	Effet
`bin/pdb status [--tables]`	état de la base et prochaines actions ; signaux de qualité des tableaux
`bin/pdb add <arxiv | doi | pdf>`	acquisition, ingestion et réindexation
`bin/pdb ingest [--force | --reparse]`	(ré)extraction des PDF de `inbox/`
`bin/pdb ingest --reparse --tables docling [--ocr]`	tableaux sans corps repris par Docling
`bin/pdb reindex`	`INDEX.md`, `external_refs.csv`, index de passages
`bin/pdb bench [filtres]`	filtre `benchmarks.csv`, colonnes de source toujours affichées
`bin/pdb bench --check | --comparable | --add-columns`	valeurs retrouvées dans leur passage ; groupes comparables ; colonnes optionnelles
`bin/pdb claims <id> [--check]`	preuves atomiques d'un article et leur contrôle
`bin/pdb paper <id | mot>...`	identité d'un ou plusieurs articles
`bin/pdb search "<q>"`	passages classés (hybride, `--rerank` en option ; `--gpu` / `--cpu`)
`bin/pdb gpu [on | off]`	état du moteur GPU ; activation pour la machine, retour au CPU
`bin/pdb show <chunk_id>`	texte exact d'un passage
`bin/pdb verify <chunk_id> "<cit.>"`	contrôle d'une citation
`bin/pdb cites | cited-by <id>`	graphe de citations internes
`bin/pdb discover`	candidats à l'ajout via OpenAlex
`bin/pdb plot [bench | eval | corpus | all]`	figures statiques et `figures/manifest.json`
`bin/pdb plot --check | --stale`	bloc `plot` proposé pour la base ; figures périmées seulement
`bin/pdb viz`	visualisation locale du corpus
`bin/pdb rag index | search | evidence | benchmarks`	moteur GPU (Qwen3) : index, recherche, preuves, chiffres
`bin/pdb rag acquire | approve | check-answer`	acquisition open access, import après accord, contrôle d'une réponse
`bin/pdb synth <étape>`	banc synthétique (corpus, génération, validation, ablations, rapport)
`bin/pdb base list | new | register | use`	bases enregistrées, création, enregistrement, base par défaut
`bin/pdb base info | check`	profil et axes d'une base ; effet des règles de `base.json`
`bin/pdb base scan | import <base>:<id>`	articles utiles d'une autre base ; copie d'un article
`bin/pdb --base <nom> <commande>`	agir sur une autre base que celle du dossier courant
`bin/pdb test`	tests unitaires
`bin/pdb eval [--extract | --rag | --speed]`	banc d'évaluation, par volet ou complet
`bin/pdb eval --save | --compare`	enregistre un rapport ; compare au dernier et aux seuils
`bin/pdb eval --rag --gpu`	recherche évaluée avec le moteur GPU, mêmes clés
`bin/pdb eval --questions | --answers <run>...`	questions seules ; notation des réponses rédigées
`bin/pdb agents [--report | --scan | --mark]`	journal et coût des sous-agents, économie de la délégation
`/paper-add`, `/paper-fiche`	skills de l'agent : ajout ; fiche et lignes de benchmark
`/paper-extract`	preuves atomiques d'un article (`claims.jsonl`)
`/paper-ask`, `/paper-gaps`, `/paper-discover`	réponse sourcée avec verdict de suffisance ; carte des lacunes ; découverte
`/eval-answers`	produit à l'aveugle les réponses d'un jeu, puis les fait noter
`/paper-research`	réponse sourcée avec le moteur GPU et

Installation. Python 3.12 et Docker (pour GROBID) :

```
bash bin/setup.sh                                  # .venv et dépendances
.venv/bin/pip install -r requirements-embed.txt    # optionnel : recherche dense
bin/pdb grobid start                               # GROBID sur localhost:8070
bin/pdb add 2507.18174 && bin/pdb test
```

TABLE 8 – Couverture des tests unitaires (nombre de fonctions de test par fichier).

Fichier	Tests	Ce qui est vérifié
`test_plot.py`	29	familles de plateformes, classes de précision, exclusion des `n/r`, bloc `plot` par base, figures périmées, rendu sur la base réelle
`test_query.py`	23	filtres, tri et agrégation de `bench`, colonnes de source forcées, `--check`, groupes comparables, identifiants courts, `paper`
`test_tables.py`	19	reconstruction des tableaux (PDF synthétique), appariement Docling, signaux de qualité, numéros de légende
`test_schema.py`	19	validateurs du modèle de données, colonnes optionnelles, hiérarchie des titres
`test_rag.py`	18	découpage (tableaux par rangées), BM25, fusion, reranking, index incrémental, moteur Qwen simulé, `verify` et contrôle des nombres
`test_evaluation.py`	18	métriques du banc sur de petits cas construits, notation des réponses finales, emplacements du banc par base
`test_tei.py`	12	parsing d'une sortie GROBID réelle
`test_openalex.py`	12	découverte, ajout par DOI et par liste (réponses simulées)
`test_base.py`	12	base active, `base.json` minimal, règles invalides, classement, bloc `plot`
`test_refs.py`	11	rapprochement des références, `external_refs.csv`, citations internes
`test_citation.py`	11	exports `.bib`/`.ris` : lecture, appariement, priorité
`test_xbase.py`	10	scan entre bases, import (en-tête de la base cible), création d'une base, onglet Bases
`test_agentlog.py`	10	journal des sous-agents : dédoublonnage, réponse incomplète, hook trop précoce relu, issue notée (`--mark`)
`test_agentcost.py`	10	coût par modèle, calage sur Claude Code, vérification, lectures dupliquées, compaction, économie estimée
`test_lineage.py`	9	généalogie sur une chaîne de citations datée
`test_graph.py`	8	graphe du corpus sur mini-corpus
`test_bench.py`	8	banc synthétique : corpus isolé, localisation des preuves, sortie structurée de `claude -p`, contrôle du modèle générateur
`test_ligatures.py`	7	réparation des ligatures perdues
`test_viz.py`	6	figures et panneau de détail, sans serveur
`test_status.py`	6	état de la base, tableaux signalés, prochaines actions
`test_ingest.py`	6	conservation des axes, tags et avertissements à la ré-extraction
`test_claims.py`	6	preuves atomiques : schéma, doublons, extrait absent, nombre altéré, mauvais passage
`test_rag_qwen.py`	5	moteur GPU en mode lexical : sources, conditions numériques, instantané périmé, acquisition isolée
`test_reindex.py`	4	notes sur les benchmarks, mesures propres
`test_embed.py`	4	cache d'embeddings, branchement dans le graphe
`test_registry.py`	3	registre des bases, ordre de résolution de la base active

TABLE 9 – Qualité de l'extraction (101 articles, tous extraits par GROBID ; entre parenthèses, le run sur 45 articles).

Métrique	Valeur
Couverture du texte : part des mots du PDF (bibliographie exclue) retrouvés dans `fulltext.md`	moy. 0,90 ; min. 0,57 (0,91 ; 0,81)
Tableaux dont le corps a été extrait (584 tableaux, dont 110 reconstruits par Docling et 11 par PyMuPDF)	0,78 (0,88)
Ligatures restantes, pour 10 000 mots	0,00
Articles dont des sections numérotées n'ont pas été détectées	31 / 101 (13 / 45)
En-tête GROBID seul contre `metadata.json` : titre / année / DOI / arXiv	0,87 / 0,47 / 0,65 / 0,82
En-tête GROBID seul : F1 sur les noms d'auteurs	0,84 (0,86)
Valeurs de `benchmarks.csv` retrouvées dans le tableau cité (1 044 valeurs)	0,70 (0,65)
idem, lignes qui ne citent que des tableaux (682 valeurs)	0,74 (0,71)
Valeurs de `benchmarks.csv` retrouvées n'importe où dans le texte	0,79 (0,76)

TABLE 10 – Recherche de passages sur 8 601 passages (101 articles, modèle bge-small-en-v1.5), le 03/10/2026 ; entre parenthèses, le recall@5 du run sur 45 articles. En gras : meilleure valeur par jeu.

Jeu	Mode	recall@5	recall@10	MRR	nDCG@10	paper@5
dev (20)	lexical (BM25)	0,85 (0,95)	0,90	0,66	0,32	0,90
	dense	**0,90** (0,90)	0,90	0,69	0,34	0,90
	hybride (défaut)	0,80 (0,90)	0,90	0,67	0,34	0,90
	hybride + reranking	**0,90** (1,00)	**0,95**	**0,76**	**0,38**	**0,95**
holdout (25)	lexical (BM25)	0,56 (0,60)	0,64	0,50	0,23	0,60
	dense	0,64 (0,64)	0,64	0,49	0,29	0,76
	hybride (défaut)	0,68 (0,72)	0,68	0,56	0,26	0,72
	hybride + reranking	**0,72** (0,80)	**0,72**	**0,62**	**0,31**	**0,80**
FR (8)	lexical (BM25)	0,00 (0,12)	0,38	0,05	0,03	0,25
	dense	**0,50** (0,62)	0,50	0,26	0,12	**0,62**
	hybride (défaut)	**0,50** (0,50)	**0,62**	**0,29**	**0,13**	**0,62**
	hybride + reranking	0,38 (0,50)	0,50	**0,29**	0,10	**0,62**

TABLE 11 – Signal « hors corpus » : part des questions signalées sur la validation (10 questions hors corpus, 25 questions du holdout), base fpga, 101 articles.

Signal	Seuil	Hors corpus signalées	Corpus signalées à tort	Adopté
cosinus, seuil initial fixé à la main	0,60	0,00	0,00	–
cosinus, seuil calibré	0,78	0,80	0,20	non
cross-encoder, seuil calibré	2,12	0,90	0,32	non
Qwen3-Reranker-4B, 28 articles (section 4.6)	0,96	1,00	0,16	non

TABLE 12 – Taux d'acceptation de `pdb verify`, avant et après le contrôle des nombres. « Avant » : run sur 28 articles ; « Après » : run du 03/10 sur 101 articles, dont le tirage aléatoire diffère (33 phrases à chiffre modifiable au lieu de 38).

Citation	Avant	Après	Statuts après
exacte (40)	1,00	1,00	40 `OK`
chiffre modifié (33)	**1,00**	**0,00**	33 `NOMBRES`
mot remplacé (40)	0,97	0,97	39 `APPROX`, 1 `NOMBRES`
d'un autre article (40)	0,00	0,00	40 `ABSENT`

TABLE 13 – Temps mesurés sur la base fpga (8 601 passages ; médianes de 3 répétitions, CPU sans GPU).

Mesure	Valeur
Extraction hors GROBID, par article (TEI → texte, ligatures, tableaux)	0,01 à 0,13 s
Chargement de l'index (8 601 passages, 17 Mo)	0,52 s
Encodage bge-small	18 passages/s ; index complet ≈ 491 s
`search` à chaud, p50 / p95 : lexical	25 / 31 ms
dense	117 / 146 ms
hybride	115 / 168 ms
hybride + reranking	1 430 / 1 782 ms
`verify` à chaud	2 ms
Commandes à froid, démarrage de Python compris : `status`	0,61 s
`bench`, `paper`	0,09 et 0,08 s
`show`	0,75 s
`search`	6,6 s

TABLE 14 – Réponses rédigées, base fpga (12 + 12 questions, recherche lexicale seule). Entre parenthèses : premier run, avant le verdict de suffisance.

Jeu	Verdict juste	Citations acceptées	Affirmations non soutenues	Sources attendues	Valeurs attendues
dev (12)	0,75 (0,83)	1,00 (0,96)	0,00 (0,02)	0,87 (0,91)	1,00 (1,00)
holdout (12)	0,83 (0,92)	1,00 (1,00)	0,00 (0,00)	0,50 (0,67)	1,00 (1,00)

TABLE 15 – Banc d'évaluation sur les deux bases qui ont des jeux de questions, le 03/10/2026 (recall@5 sauf mention).

Mesure	fpga (101 articles)	drone (87 articles)
Passages	8 601	5 361
Couverture du texte : moyenne / minimum	0,90 / 0,57	0,91 / 0,77
Tableaux avec corps	0,78	0,79 (0,90 sur les 73 articles GROBID)
En-tête : titre / année / DOI	0,87 / 0,47 / 0,65	0,92 / 0,64 / 0,70
Valeurs de `benchmarks.csv` retrouvées dans le tableau cité	0,70 (1 044)	0,62 (739)
Dev, hybride / hybride + reranking	0,80 / 0,90	0,90 / 1,00
Réservé, hybride / hybride + reranking	0,68 / 0,72 (25 q.)	0,90 / 0,95 (20 q.)
Réservé, MRR hybride + reranking	0,62	0,89
Français, hybride + reranking	0,38	0,50
Pertinence faible (`cos`) : hors corpus / corpus signalés	0,80 / 0,20	1,00 / 0,10
`verify` : exacte / mot remplacé / chiffre modifié / autre article	1,00 / 0,97 / 0,00 / 0,00	1,00 / 0,82 / 0,03 / 0,00